Project Package Structure

Primary classes and responsibilities by Python module

neat/

NEAT – `neat.py`
`initialize_population()`
`evaluate_population()`
`speciate()`
`reproduce()`
`evolve()`

Genome – `genome.py`
`mutate()`
`mutate_weights()`
`mutate_structure()`
`crossover()`
`compatibility_distance()`
`validate()`
`serialize()`
`GenomeConfig: dataclass`

Species – `species.py`
`add()`
`remove()`

`genes.py`
`NodeGene: dataclass`
`ConnectionGene: dataclass`

Innovation – `innovation.py`
`get_cell_innovation()`
`get_connection_innovation()`

models/

Model – `model.py`
`forward()`
`reset_state()`
`get_state()`
`set_state()`

Compiler – `compiler.py`
`compile_genome()`
`validate_topology()`
`build_execution_plan()`

Cells – `cells.py`
FFNN, RNN, and LSTM tensor operations and execution coordination.

simulations/

Simulation – `simulation.py`
`run_sim()`
`reset()`
`get_observations()`
`step()`
`is_done()`
`evaluate()`
`set_trajectory_method()`
get_trajectory()

MazeSimulation – `maze.py`
`reset()`
`get_observations()`
`step()`
`is_done()`
`evaluate()`

PoleBalancingSimulation
`pole_balancing.py`
`reset()`
`get_observations()`
`step()`
`is_done()`
`evaluate()`

visualization/

TopologyVisualization – `topology.py`
`plot_genome()`
`draw_nodes()`
`draw_connections()`

Replay – `replay.py`
`load_episode()`
`replay()`
`render()`

experiments/

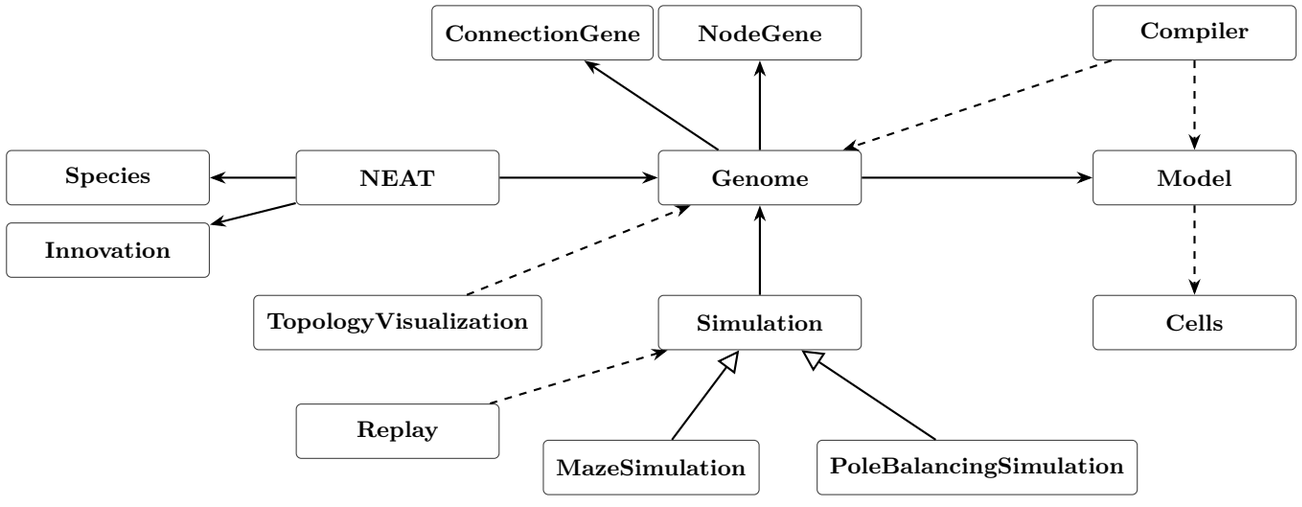
Training – `train.py`
Experiment setup and evolutionary training entry point.

Evaluation – `evaluate.py`
Evaluate saved genomes and compare results.

`configs/`
Experiment-specific configuration files.

Class Relationships

Conceptual ownership, inheritance, and usage relationships



Relationship legend

- Solid arrow: A owns or maintains instances of B as part of its state.
- Dashed arrow: A uses B's functionality without owning its instances.
- Open triangle: A inherits from B; the triangle points to the parent.